

Rapport: AutoBattler

1. Choix de conception

Dès le début de la conception de notre programme, nous nous sommes fixés plusieurs règles. Notre objectif était de maintenir un code clair, et facile à faire évoluer, car nous avons commencé par développer les fonctionnalités obligatoires, pour ensuite y ajouter nos nouvelles fonctionnalités. Par exemple, le fichier `Jeu.java` a été développé en parallèle de nombreuses fonctionnalités, afin de pouvoir tester ces nouvelles méthodes sans difficultés.

Le travail en binôme nous a forcé à garder un code compréhensible et bien organisé, pour que l'on puisse reprendre le travail de l'autre facilement via GitHub. Notre autre intérêt était aussi de rester le plus fidèle possible au cahier des charges donné pour ce projet.

À un certain moment, le nombre de fichiers devenant trop important, la navigation devenait compliquée. C'est pourquoi nous avons regroupé les classes en packages selon leur utilité. Par exemple, tout le code est contenu dans `src`, et les personnages (combattants, boss, mini-boss, mobs) sont rangés dans le package `entitees`. Le package `items` regroupe tous les items utilisables par le joueur; inventaire gère les inventaires du joueur et du marchand (utile pour le jeu alternatif); enfin, le package `jeu` contient tout ce qui est lié à la logique du jeu (gestion des événements, combats, équipes, fonctions utilitaires...). Cette séparation en différents dossiers (packages) triés par responsabilité a permis de rendre la navigation dans le gestionnaire de fichier beaucoup plus simple, et permet d'ajouter de nouvelles classes avec plus de facilités.

L'entrée du programme est la classe `Launcher.java`, qui permet de lancer soit le jeu classique (`Jeu.java`), soit le jeu alternatif (`JeuAlternatif.java`). On a choisi de séparer ces deux modes lorsqu'on s'est rendu compte que le fonctionnement du jeu alternatif était assez lourd et différent des modes de jeux proposés par `Jeu.java`. Pour gérer tous les types de combattants (Guerrier, Mage, etc.), on a utilisé une classe abstraite `Combattant`, qui définit les comportements de base communs à tous les combattants, mobs, boss, miniboss. Grâce à l'héritage et au polymorphisme, chaque type de personnage peut avoir son propre comportement (contre-attaque, soin, esquive...) tout en héritant de la classe mère, ce qui simplifie énormément l'ajout de nouveaux personnages.

L'inventaire repose aussi sur une classe abstraite `Inventaire`, qui utilise une `HashMap<Item, Integer>` pour gérer la quantité de chaque items possédés, ce qui est plutôt utile pour un inventaire. Plutôt que de surcharger cette classe avec trop de logique, on a préféré créer deux sous-classes : `InventaireJoueur` et `InventaireMarchand`. La classe `inventaireMarchand` est ici pour gérer les achats, les ventes ou encore l'affichage de l'inventaire du marchand.

Du côté des items, nous avons décidé de développer une classe abstraite `Item`, avec des sous classes définissant les items concrets (`Potion`, `Bière`, `Thèse`, etc.). Cette structure rend l'ajout d'un nouvel objet très simple, sans modifier le reste du programme : il suffit de créer une nouvelle sous-classe. De plus on a fait le choix qu'en précisant le nouvel item dans la classe mère, celui ci peut directement être utilisé en jeu (vendu par le marchand).

Nous avons décidés de séparer le package jeu en trois sous package qui sont les suivants:
Le package mode regroupe les différents modes de jeu (PvE, PvP, test, alternatif) et leurs affichages.

Le package gestions contient des classes nécessaires au bon fonctionnement du jeu, des combats et des équipes. Par exemple GestionEvenement, gère les événements spéciaux pour le jeu alternatif (marchand, repos, combats), ou UtilitaireJeu, qui regroupe des fonctions communes comme la saisie utilisateur. Cela permet de ne pas surcharger les fichiers Jeu.java et JeuAlternatif.java.

Le package comparateurs contient plusieurs classes Comparator servant à trier les combattants selon différents critères (PV, vitesse). Ces comparateurs sont utilisés dans plusieurs contextes : ordre d'attaque, pour la fonction soin, ou encore attaques de cibles spéciales.

2. Fonctionnalités implémentés

Entitees

Class Combattant

Combattant() → Création du combattant

void cibleAdverse(equipe c) → Choisis une cible aléatoire parmi les ennemis

void action(Equipe ennemie, Equipe alliée) → La classe choisie l'action qu'il va réaliser (ex : attaque ou soigner)

void attaquer(Equipe ennemie) → Permet au combattant d'attaquer une cible choisie

void prendreDegat(Combattant attaquant) → Si le combattant subit des dégâts la méthode est appelée, il perd de la vie, du courage et peut éventuellement esquiver

int calculerDegat(Combattant cible) → Calcul les dégâts que le combattant va infliger à sa cible

void regenererPV(int pv) → Régénère les pv du combattant

void regenererCourage(int courage) → régénère le courage du combattant

void soustraireCourage(int courage) → réduis le courage du combattant

boolean activerEsquive() → En fonction du tirage la fonction renvoie vrai si le combattant a esquivé sinon faux

boolean estEnVie() → Renvoie vrai si le combattant est vivant sinon faux

String mortOuVivant() → Renvoie une chaîne de caractère en fonction de si le combattant est vivant, mort ou s'est enfui

Combattants

Class Abomination

@Override

void attaquer(Equipe ennemis) → Plus le courage de l'abomination est faible plus il attaque (de 0 à -30, +1 attaque par -10 de courage)

Class Archer

@Override

void attaquer(Equipe ennemis) → L'archer attaque la cible avec le moins de pv

Class Berserker

@Override

void attaquer(Equipe ennemis) → Si le berserker est en dessous d'un seuil il attaque à nouveau

Class Guerrier

@Override

void prendreDegat(Combattant cible) → lorsque que le combattant prend des dégâts il peut contre-attaquer

Class Mage

@Override

int calculerDegat() → permet au mage d'ignorer l'armure

Class Paladin

@Override

void attaquer(Equipe ennemis) → permet au paladin de se soigner pendant son tour

Class Pretre

void soin(Equipe cible) → le prêtre peut soigner ses alliés

@Override

void action(Equipe ennemie, Equipe allié) → Le prêtre choisi entre attaquer ou soigner

Class Voleur

@Override

boolean activerEsquive() → le voleur à 50% de chance d'esquiver

Boss

Class des boss

Les boss ont des méthodes et attributs identiques

@Override

void attaquer(Equipe ennemis) → Les boss attaquent 4 fois

Mini-boss

Class des minis-boss

Les minis-boss ont des méthodes et attributs identiques

@Override

void attaquer(Equipe ennemis) → Les minis-boss attaquent 2 fois

Mobs

Class des mobs

Les mobs ont simplement des stats qui varient entre eux

Inventaire

Class Inventaire

Inventaire() → Création d'un Inventaire

void starterKit() → Donne à l'inventaire des ressources dès le début du jeu

void utiliserItem(Item item, Equipe equipe) → Permet d'utiliser des items

boolean utiliserItemNom(String nom, Equipe equipe) → Utilise un item en fonction de son nom

void ajoutItem(Item item, int nombre) → Permet d'ajouter des items à l'inventaire

boolean supprimerItem(Item item, int nombre) → Permet de supprimer un ou plusieurs items

void ajoutOr(int nombre) → Ajoute de l'or

void soustraireOr(int nombre) → Reduis l'or

String toString() → Affiche l'inventaire

Class dérivées de Inventaire

Class InventaireJoueur

void venteJoueur(Item item, int nombre) → Permet au joueur de vendre ses items

@Override

void starterKit() → Donne au joueur des ressources lorsque qu'il lance la partie

Class InventaireMarchand

void achatMarchand(Item item, int nombre, InventaireJoueur inventaireJoueur) → permet d'acheter des items auprès du marchand

void venteMarchand(Item item, int nombre, InventaireJoueur inventaireJoueur) → permet de vendre ses items auprès du marchand

void starterKit() → Génère un magasin avec des ressources aléatoires

String toString() → Affichage de l'inventaire

Items

Class Item

void effet(Equipe equipe) → Donne aux items différents effets

Class dérivée de Item

Bière

Redonne 50 de courage à tous les Combattants

Potion

Régénère tous les combattants de 50 pv.

PorteurdeCendre, These

Affiche du texte

Jeu

Compareurs

Class Comparator

ComparatorVie()

Permet de comparer les la vie des combattants de façon décroissante

ComparatorVitesse()

Permet de comparer la vitesse des combattants de façon décroissante

Gestion

Class Combat

Combat(Equipe equipe_1, Equipe equipe_2, Scanner scanner) → Initialise le combat

void lancerCombat() → Permet de lancer le combat

void lancerMache() → Lance une manche

String resultatManche() → Affiche le resultat de la manche

String ecranVictoire() → Affiche l'écran de victoire

Class Equipe

Equipe() → Créer une liste pour constituer l'équipe

Combattant choisirCombattantAleatoire() → Permet au combattant de choisir un combattant aléatoire

Combattant choisirCombattantFaible() → Permet au combattant de choisir l'ennemi avec le moins de pv

boolean aDesVivants() → Permet de savoir si toute l'équipe est morte ou non

Class GenCombattant

Combattant generer(int choix) → Génère un combattant en fonction du nombre en paramètre

Combattant genererParNom(String nom) → Génère un combattant en fonction de nom

void afficherMenuAlt() → Affiche les combattants qui peuvent potentiellement être générés

Class GestionEvenement

GestionEvenement(Scanner scanner, Equipe equipe, InventaireJoueur inventaire) → Permet de gérer les evenements que le joueur va rencontrer

void genereMobs(Equipe groupe) → Génère un groupe aléatoire de monstre e fonction d'un tirage

void genererMiniBoss(Equipe boss) → Génère un mini-boss aléatoire en fonction du tirage

void genererBoss(Equipe boss) → Génère un boss en fonction du tirage

void repos() → En fonction de son choix le joueur peut soit régénérer sa vie ou son courage

void eventMarchand() → Gère les interactions ainsi que les dialogues avec le marchand

void tableDeButin() → Gère le taux de drop des items

void prochainEvent() → Définis le prochain événement que le joueur va rencontrer aléatoirement ou en fonction de la manche pour les boss/minis-boss

void arreterJeu() → Permet d'arrêter la partie

void ajoutOr() → s'utilise en fin de combat pour déterminer combien d'or le joueur a obtenu ainsi que les items potentiellement drop

UtilitaireJeu

Gère la saisie du joueur dans les menus

Mode

Jeu

void choixCombattantTest(Equipe equipe, Scanner scanner) → Demande au joueur les combattants qu'il veut ajouter dans son equipe pour le mode test

void choixCombattantPvp(Equipe equipe, Scanner scanner) → Propose des combattants à ajouter pour son équipe dans le mode pvp

void choixCombattantOrdi(Equipe equipe) → Equipe généré aléatoirement pour l'ordinateur

boolean demanderQuitter(Scanner scanner) → Permet de quitter le jeu

void lancerJeu() → Lance/ferme le menu du jeu et démarre le mode de jeu sélectionné

Jeu Alternatif

void choixCombattant(Equipe equipe, Scanner scanner) → Le joueur choisit ses combattants (4 max)

void boucleDeJeu(Scanner scanner, Equipe equipe, Inventaire joueur inventaire) → Entre chaque round le joueur choisit la prochaine action qu'il veut effectuer dans le menu

void lancerJeuAlternatif() → Permet de lancer le mode de jeu alternatif

Launcher

Le joueur choisit si il veut joué à la version originale du jeu, accéder au wiki ou a l'extension

3. Fonctionnement global

Le joueur exécute le launcher et sélectionne la version du jeu à laquelle il veut jouer.

1. Lancement du jeu original

Le joueur choisit entre 3 modes de jeu :

A. Mode test

Le joueur choisit la composition des deux équipes, les équipes s'affrontent et le vainqueur est affiché à la fin .

B. Mode PVP.

Les joueurs constituent chacun à leur tour une équipe. À chaque tour, le joueur qui choisit sélectionne parmi 3 personnages le combattant qu'il veut ajouter à son équipe (5 fois par joueur). Le combat se lance et le vainqueur est affiché à la fin

C. Mode PVE

Le joueur constitue son équipe de la même manière qu'en pvp, l'équipe adverse est déterminé aléatoirement.

Le combat se lance et le vainqueur est affiché à la fin.

2. Lancement du jeu alternatif

Le mode de jeu s'inspire d'un dungeon crawler, le joueur doit aller le plus loin possible tout en gérant la vie de ses personnages ainsi que leur jauge de courage.

Au lancement du jeu, le joueur sélectionne 4 combattants à ajouter à son équipe

Le joueur doit ensuite choisir ce qu'il veut faire :

A. Prochain event :

Lance un évènement aléatoire :

* 7 chance sur 10 de se retrouver dans un combat

Le combat est généré aléatoirement, le groupe ennemi peut avoir entre 3 et 6 monstres

* 2 chance sur 10 de rencontrer un marchand

Le joueur peut acheter ou vendre des consommables auprès du marchand

Les items ont une moins bonne valeur lorsqu'ils sont vendus par le joueur

* 1 chance sur 10 de trouver une zone de repos

Le joueur choisit entre régénérer le courage ou la vie de son équipe.

- * Toutes les 5 manches un mini-boss apparait.

- * Toutes les 10 manches un boss apparait.

B. Inventaire :

Ici le joueur peut consulter son inventaire et utiliser ses consommables, les items importants sont les potions et bières

- * **Potion** : régénère la vie de toute l'équipe de 50
- * **Biere** : régénère le courage de toute l'équipe 50

C. Consulte l'équipe :

Permet de visualiser l'équipe actuelle

D. Quitter :

Quitte le jeu, retour au menu du launcher.

3. **Wiki**

Ouverture du wiki (Lien externe).

[\[Lien vers le wiki\]](#)

4. **Quitter**

Quitte le launcher (ferme le programme).

4. Diagramme UML

Page suivante.

<<Abstract>> Combattant	
- in pv	
- int pvMax	
- int courage	
- int attaque	
- int defense	
- int vitesse	
- int fuite	
- static int idCpt	
- final int id	
Combattant(int pvMax, int attaque, int defense, int vitesse, int courageMax)	
+ void action(Equipe ennemie, Equipe alliee)	
+ void attaquer(Equipe ennemis, Equipe alliee)	
+ void prendreDegat(Combattant attaquant)	
+ int calculerDegat(Combattant cible)	
+ void regenererPV(int pv)	
+ void regenererCourage(int courage)	
+ void soustraireCourage(int courage)	
+ boolean activerEsquive()	
+ boolean estEnVie()	
+ boolean mortOuVivant()	
+ int getPvMax()	
+ int getPv()	
+ int getAttaque()	
+ int getDefense()	
+ int getVitesse()	
+ int getId()	
+ int getCourageMax()	
+ int getCourage()	
+ abstract String toString()	
+ abstract String getNom()	

package combattants

Abomination
extends Combattant
+ Abomination()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Archer
extends Combattant
+ Archer()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Paladin
extends Combattant
+ Paladin()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Berserker
extends Combattant
+ Berserker()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Guerrier
extends Combattant
+ Guerrier()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Voleur
extends Combattant
+ Voleur()
+ boolean activerEsquive()
+ String toString()
+ String getNom()

Mage
extends Combattant
+ Mage()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ int calculerDegat(Combattant cible)
+ String toString()
+ String getNom()

Pretre
extends Combattant
+ Pretre()
+ void soin(Equipe cible)
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

package miniboss

Graymarrow
extends Combattant
+ Graymarrow()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Herald
extends Combattant
+ Herald()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

KingSlime
extends Combattant
+ KingSlime()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

package boss

Sanguinius
extends Combattant
+ Sanguinius()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Ragnaros
extends Combattant
+ Ragnaros()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Monika
extends Combattant
+ Monika()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

Nashor
extends Combattant
+ Nashor()
+ void attaquer(Equipe ennemis, Equipe alliee)
+ String toString()
+ String getNom()

package mobs

Diablotin
extends Combattant
+ Diablotin()
+ String toString()
+ String getNom()

Cultist
extends Combattant
+ Cultist()
+ String toString()
+ String getNom()

Demon
extends Combattant
+ Demon()
+ String toString()
+ String getNom()

package entitees

package src

Launcher
+ static void main(String[] args)

package jeu

package comparator

ComparatorVitesse
implements Comparator<Combattant>
+ int compare(Combattant o1, Combattant o2)

ComparatorVie
implements Comparator<Combattant>
+ int compare(Combattant o1, Combattant o2)

package mode

Jeu
static void choixCombattantTest(Equipe equipe, Scanner scanner)
+ static void choixCombattantPvp(Equipe equipe, Scanner scanner)
+ static void choixCombattantOrdi(Equipe equipe)
+ static void lancerJeu()

JeuAlternatif
+ static void choixCombattant(Equipe equipe)
+ static void boucleDeJeu(Equipe equipe, InventaireJoueur inventaire)
+ static void lancerJeuAlternatif()

UtilitaireJeu
+ static final Scanner scanner
+ static int saisieMenu(int min, int max)
+ static int saisirInt()

package gestion

Combat
- Equipe equipe1
- Equipe equipe2
- int compteurManche
+ Combat(Equipe equipe1, Equipe equipe2)
+ void lancerCombat()
+ void lancerManche()
+ String resultatManche()
+ String ecranVictoire()

GenCombattant
- static final List<String> noms
+ static List<String> genNomCombattants()
+ static Combattant generer(int choix)
+ static Combattant genererParNom(String nom)
+ static void afficherMenuAlt()
+ static int getNbCombattants()

GestionEvenement
- Equipe equipe
- InventaireJoueur inventaire
- int compteur
+ GestionEvenement(Equipe equipe, InventaireJoueur inventaire)
+ void genererMobs(Equipe groupe)
+ void genererMiniBoss(Equipe boss)
+ void genererBoss(Equipe boss)
+ void evenementRepos()
+ void evenementMarchand()
+ void genererTableDeButin()
+ void prochainEvenement()
+ void arreterJeu()
+ void ajoutOr()
+ int getCompteur()

Equipe
- List<Combattant> equipe
- static int nbEquipe=1
- int id
+ Equipe()
+ Combattant choisirCombattantAleatoire()
+ Combattant choisirCombattantFaible()
+ boolean aDesVivants()
+ List<Combattant> getEquipe()
+ String toString()
+ void ajouterCombattant(Combattant c)
+ int getId()

package inventaire

<<Abstract>> Inventaire	
- int or	
- HashMap<Item,Integer>,inventaire	
+ Inventaire()	
+ void starterKit()	
+ void utiliserItem(Item item, Equipe equipe)	
+ boolean utiliserItemNom(String nom ,Equipe equipe)	
+ void ajoutItem(Item item, int nombre)	
+ boolean supprimeItem(Item item, int nombre)	
+ void ajoutOr(int nombre)	
+ void soustraireOr(int nombre)	
+ String toString()	
+ int getOr()	
+ HashMap<Item, Integer> getInventaire()	

InventaireMarchand
extends Inventaire
+ InventaireMarchand()
+ void achatMarchand(Item item, int nombre, InventaireJoueur inventaireJoueur)
+ void venteMarchand(Item item, int nombre, InventaireJoueur inventaireJoueur)
+ void starterKit()
+ String toString()

InventaireJoueur
extends Inventaire
+ InventaireJoueur()
+ void venteJoueur(Item item, int nombre)
+ void starterKit()

package items

<<Abstract>> Item	
- String nom	
- String type	
- int valeurRevente	
- int valeurVente	
+ Item(String nom, String type, int valeurVente, int valeurRevente)	
+ abstract void effet(Equipe equipe)	
+ boolean equals(Object o)	
+ int hashCode()	
+ String toString()	
+ static List<Item> getItemDisponibles()	
+ int getValeurRevente()	
+ int getValeurVente()	
+ String getNom()	
+ String getType()	

Biere
extends Item
- final int regenerePeur
+ Biere()
+ void effet(Equipe equipe)
+ String toString()

Potion
extends Item
- final int ajoutePv
+ Potion()
+ void effet(Equipe equipe)
+ String toString()

These
extends Item
+ These()
+ void effet(Equipe equipe)
+ String toString()

PorteurdeCendre
extends Item
+ PorteurdeCendre()
+ void effet(Equipe equipe)